

RETAILMART V3 • SQL

# Conditional Logic & Derived Columns

## WhatsApp Announcement

### CONDITIONAL LOGIC & DERIVED COLUMNS

-----  
What if your query needs to make DECISIONS? 'If salary > 50000, label as Senior. If NULL, show N/A. Convert text to number.' Today you learn CASE WHEN, COALESCE, NULLIF, and CAST — the decision-making tools of SQL.  
-----

Today's Focus:

- CASE WHEN — IF-ELSE logic inside SQL queries
- COALESCE — replace NULLs with defaults
- NULLIF — create NULL conditionally
- CAST & :: — convert between data types

After today, your queries can think and decide!

— Siraj Sir

#Day7 #CASEWHEN #PostgreSQL

# TODAY'S AGENDA (2 Hours)

---

| Section                           | Time    | Topics                                                                                                                                                                                                                                          |
|-----------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quick Recap — earlier sessions    | 5 mins  | String & Date functions from Scalar Functions, Scalar functions = transform one value                                                                                                                                                           |
| Part 1: CASE WHEN — SQL's IF-ELSE | 50 mins | Simple CASE (value matching — like switch), Searched CASE (condition-based — like if/else if), CASE in SELECT, WHERE, ORDER BY, CASE inside Aggregates (conditional counting), 4 practice problems (Easy → Crazy), 3 Try It Yourself challenges |
| Part 2: COALESCE, NULLIF & CAST   | 55 mins | COALESCE — first non-NULL value + practice, NULLIF — conditionally create NULL + practice, CAST & :: — type conversion + practice, Combined hard & crazy problems (all 4 tools), 2 Try It Yourself challenges                                   |
| Summary & Wrap-up                 | 30 mins | Common mistakes gallery, Interview bank — conditional logic, Syntax Reference Card, Homework + Aggregates & Grouping preview                                                                                                                    |

# YOUR SQL JOURNEY

- ✓ SQL Intro
- ✓ Setup & RetailMart
- ✓ DDL & DML
- ✓ Data Types + Constraints
- ✓ Filtering & Sorting
- ✓ Scalar Functions (String & Date)
- 7** **Conditional Logic & Derived Columns**

← HERE

# Memory Check

earlier sessions: SQL categories, PostgreSQL setup, RetailMart V3.

earlier sessions: DDL, Data Types, Constraints, DML.

SELECT, WHERE, AND/OR, LIKE, BETWEEN, IN, IS NULL, ORDER BY, LIMIT.

String functions (UPPER, TRIM, CONCAT, SPLIT\_PART) + Date functions (NOW, EXTRACT, AGE, TO\_CHAR, DATE\_TRUNC).

Today: We can FIND and TRANSFORM data. Now we add DECISION-MAKING — different output based on conditions.

# The Report Card Generator

A school needs to generate 2,000 report cards. Each student has raw marks (0-100). The system needs to:

- Convert marks to grades: A+ (90+), A (80-89), B (70-79), C (60-69), F (below 60)
- Handle absent students (NULL marks) — show 'ABSENT' not blank
- Display percentage as text with % symbol
- Avoid crash when total\_students = 0 (division by zero)

This is EXACTLY what CASE WHEN, COALESCE, NULLIF, and CAST do.

## From Story to SQL!

Convert marks to grades → `CASE WHEN marks >= 90 THEN 'A+'...`

Handle absent students → `COALESCE(marks, 'ABSENT')`

Format percentage → `CAST(marks AS TEXT) || '%'`

Avoid division by zero → `total / NULLIF(divisor, 0)`

`CASE WHEN` is the most powerful expression in SQL. It turns raw data into business-meaningful categories.

# The Traffic Signal Analogy

Red → STOP. Yellow → SLOW DOWN. Green → GO.

A traffic signal checks the value and gives a result. CASE WHEN works the same way — it checks conditions and returns the first matching result. Like IF-ELSE in programming, but inside your SQL query.



## DEFINITION

# CASE WHEN

Technical:

CASE evaluates conditions top-to-bottom and returns the result of the FIRST matching condition. Two forms: Simple CASE (matches one expression against values) and Searched CASE (evaluates boolean conditions). If no match and no ELSE, returns NULL.

Layman's:

CASE WHEN is like a series of IF-ELSE rules. 'If the order is above 10,000 then VIP. If between 5,000-10,000 then Regular. Otherwise Budget.' SQL checks each rule in order and uses the first one that matches.

## SYNTAX: Simple CASE

```
-- Simple CASE: match one column against values
-- Like a switch statement in programming
```

```
SELECT column_name,
       CASE column_name
         WHEN 'value1' THEN 'result1'
         WHEN 'value2' THEN 'result2'
         WHEN 'value3' THEN 'result3'
         ELSE 'default_result'
       END AS alias_name
FROM table_name;
```

```
-- Key rules:
-- 1. Checks top-to-bottom, first match wins
-- 2. ELSE catches everything not matched
-- 3. Without ELSE, unmatched rows get NULL
```

PRO TIP

## PRO TIP

Simple CASE is best when you are matching ONE column against EXACT values — like status codes, category names, or tier labels. If you need ranges (price > 1000) or multiple column checks, use Searched CASE instead.

## SYNTAX: Searched CASE

```
-- Searched CASE: evaluate boolean conditions  
-- Like if/else if/else in programming
```

```
SELECT column_name,  
       CASE  
         WHEN condition1 THEN 'result1'  
         WHEN condition2 THEN 'result2'  
         WHEN condition3 THEN 'result3'  
         ELSE 'default_result'  
       END AS alias_name  
FROM table_name;
```

```
-- Key: conditions can use any operator  
-- >, <, >=, BETWEEN, LIKE, IN, IS NULL, etc.
```

## Condition Order Matters!

The Mistake:

CASE WHEN price > 1000 THEN 'Budget' WHEN price > 10000 THEN 'Mid-Range' END — a product with price 50000 matches the FIRST condition (> 1000) and gets labeled 'Budget'!

The Fix:

Always put the MOST SPECIFIC condition first: WHEN price > 50000 ... WHEN price > 10000 ... WHEN price > 1000.  
Check from highest to lowest.

## CASE in WHERE — Conditional Filtering

```
-- Show only 'problem' orders based on rules
SELECT order_id, order_status, net_total,
       TO_CHAR(order_date, 'DD-Mon-YYYY') AS dt
FROM sales.orders
WHERE
  CASE
    WHEN order_status = 'Delivered'
      AND net_total > 20000
    THEN 'flag'
    WHEN order_status = 'Processing'
      AND order_date < '2025-06-01'
    THEN 'flag'
    WHEN order_status = 'Failed'
    THEN 'flag'
  END = 'flag'
ORDER BY order_date LIMIT 15;

-- CASE in WHERE = complex multi-condition
-- filtering that AND/OR alone cannot do
```

## CASE in ORDER BY — Custom Sort Priority

```
-- Sort orders by urgency, not alphabetically
-- 'Processing' first, 'Failed' last
SELECT order_id, order_status, net_total
FROM sales.orders
ORDER BY
    CASE order_status
        WHEN 'Processing' THEN 1
        WHEN 'Out for Delivery' THEN 2
        WHEN 'Shipped' THEN 3
        WHEN 'Delivered' THEN 4
        WHEN 'Returned' THEN 5
        WHEN 'Cancelled' THEN 6
        WHEN 'Failed' THEN 7
        ELSE 8
    END,
    order_date ASC
LIMIT 20;
```

PRO TIP

## PRO TIP

CASE in ORDER BY is extremely useful for dashboards. Users expect urgent items at the top — not alphabetical sorting. Assign numeric priorities (1 = most urgent) and ORDER BY that CASE expression ascending.



# The Dashboard Single-Query Trick

Management wants ONE query that shows: total orders, delivered count, pending count, cancelled count. Without CASE inside aggregates, you would need 4 separate queries. With CASE inside COUNT, one query does it all.

## SYNTAX: CASE Inside COUNT / SUM / AVG

-- Count only rows matching a condition

```
COUNT(CASE WHEN condition THEN 1 END)
```

-- Sum only matching amounts

```
SUM(CASE WHEN condition THEN amount END)
```

-- Average only matching values

```
AVG(CASE WHEN condition THEN value END)
```

-- How it works:

-- CASE returns 1 (or the value) for matches

-- CASE returns NULL for non-matches

-- COUNT/SUM/AVG skip NULLs automatically!

PRO TIP

## PRO TIP

CASE inside aggregates is a Aggregates & Grouping preview — it combines conditional logic (Conditional Logic) with aggregate functions. This pattern appears in almost every analytics dashboard. Master it and you can build any summary report in a single query.

## Easy: Customer Tier Labels

EASY

SCENARIO: The marketing team needs customer tiers displayed with friendly labels for the email campaign system.

TASK: Use Simple CASE on `customers.customers.tier` to map each tier to a display label: 'Gold' → 'Premium Member', 'Silver' → 'Valued Customer', 'Bronze' → 'Active Shopper', anything else → 'New Customer'. Show `customer_id`, name (INITCAP), tier, and the display label. Show 15 customers.

HINT: Two CASE forms exist — which one fits exact-value matching against a single column? Trap: don't forget an ELSE branch for unhandled values, or those rows will return NULL instead of a label.

## Answer

✓ ANSWER

```
SELECT customer_id,  
       INITCAP(first_name) || ' ' ||  
       INITCAP(last_name) AS name,  
       tier,  
       CASE tier  
         WHEN 'Gold' THEN 'Premium Member'  
         WHEN 'Silver' THEN 'Valued Customer'  
         WHEN 'Bronze' THEN 'Active Shopper'  
         ELSE 'New Customer'  
       END AS display_tier  
FROM customers.customers  
LIMIT 15;
```

## Medium: Employee Salary Band Classification

MEDIUM

SCENARIO: HR needs employees classified into salary bands for a compensation review: 'Executive' (salary > 100000), 'Senior' (60000-100000), 'Mid-Level' (30000-60000), 'Entry' (below 30000).

TASK: Use Searched CASE on stores.employees.salary. Show employee\_id, name, role, salary, and salary\_band. Order by salary descending. Show 20 employees.

HINT: Two CASE forms exist — which one supports range checks (>=, <)? Trap: order matters in WHEN clauses. The first match wins — if you put the lowest range first, higher values still fall into it.

## Answer

✓ ANSWER

```
SELECT employee_id,  
       INITCAP(first_name) || ' ' ||  
       INITCAP(last_name) AS name,  
       role, salary,  
       CASE  
         WHEN salary > 100000 THEN 'Executive'  
         WHEN salary > 60000 THEN 'Senior'  
         WHEN salary > 30000 THEN 'Mid-Level'  
         ELSE 'Entry'  
       END AS salary_band  
FROM stores.employees  
ORDER BY salary DESC  
LIMIT 20;
```

# Hard: Shipment Delivery Speed Rating

HARD

SCENARIO: The logistics team wants to rate deliveries by speed. If delivered within 3 days of shipping → 'Fast'. Within 7 days → 'Normal'. More than 7 days → 'Slow'. Not yet delivered → 'In Transit'.

TASK: Use Searched CASE on the day difference between delivered\_date and shipped\_date. Show shipment\_id, courier\_name, shipped\_date, delivered\_date, days taken, and the speed rating. Show 20 shipments.

HINT: delivered\_date and shipped\_date are DATE columns, so delivered\_date - shipped\_date is already an integer number of days — compare it directly. Trap: when delivered\_date IS NULL, the subtraction returns NULL and your bucket logic fails — handle the NULL case as the FIRST WHEN in your CASE.



## Answer



ANSWER

```
SELECT shipment_id,  
       UPPER(courier_name) AS courier,  
       TO_CHAR(shipped_date, 'DD-Mon-YYYY')  
       AS shipped,  
       TO_CHAR(delivered_date, 'DD-Mon-YYYY')  
       AS delivered,  
       CASE  
         WHEN delivered_date IS NULL  
           THEN 'In Transit'  
         WHEN delivered_date - shipped_date <= 3  
           THEN 'Fast'  
         WHEN delivered_date - shipped_date <= 7  
           THEN 'Normal'  
         ELSE 'Slow'  
       END AS speed_rating  
FROM sales.shipments  
ORDER BY shipped_date DESC  
LIMIT 20;
```

# Crazy: Payroll Tax Bracket Assignment

CRAZY

SCENARIO: The CFO wants to verify the income tax slabs on payroll.pay\_slips. For each employee's gross salary, classify into Indian income-tax brackets: 0% ( $\leq 2.5L$ ), 5% ( $\leq 5L$ ), 20% ( $\leq 10L$ ), 30% ( $> 10L$ ). This is exactly the range-based CASE WHEN pattern.

TASK: From payroll.pay\_slips, classify gross\_salary into bracket labels. Show employee\_id, salary\_month, gross\_salary, the expected tax\_rate, and a column comparing 'Expected vs Actual' income\_tax. Filter to most recent pay slips.

HINT: Range buckets are a Searched CASE — each WHEN tests an inequality, ORDERED from smallest to largest. Trap: Indian tax brackets use ANNUAL income but pay\_slips show MONTHLY gross — multiply gross\_salary by 12 before comparing to the annual thresholds, or your buckets misclassify everyone.

## Answer

✓ ANSWER

```
SELECT
    employee_id,
    salary_month,
    gross_salary,
    gross_salary * 12 AS annual_estimate,
    CASE
        WHEN gross_salary * 12 <= 250000
            THEN '0% (Nil Bracket)'
        WHEN gross_salary * 12 <= 500000
            THEN '5% (Lower)'
        WHEN gross_salary * 12 <= 1000000
            THEN '20% (Middle)'
        ELSE '30% (Top)'
    END AS expected_bracket,
    income_tax AS actual_tax
FROM payroll.pay_slips
ORDER BY payment_date DESC
LIMIT 15;
```

## Try It Live!

Run each query on your RetailMart database. Modify the conditions. Add your own categories. Understanding comes from experimentation.

## Challenge 1: Weekend vs Weekday Orders

```
-- Label each order as 'Weekend' or 'Weekday'
SELECT order_id, order_date,
       TO_CHAR(order_date, 'Day') AS day_name,
       CASE
         WHEN EXTRACT(DOW FROM order_date)
              IN (0, 6) THEN 'Weekend'
         ELSE 'Weekday'
       END AS day_type
FROM sales.orders
LIMIT 15;

-- YOUR TURN: Try adding 'Friday Evening'
-- as a separate category
```

## Challenge 2: Product Availability Flag

```
-- Flag products based on inventory
SELECT p.product_id, p.product_name,
       p.price,
       CASE
         WHEN p.price > 50000 THEN 'Premium'
         WHEN p.price > 5000 THEN 'Standard'
         ELSE 'Economy'
       END AS segment
FROM products.products p
ORDER BY p.price DESC LIMIT 15;

-- YOUR TURN: Add a 'Luxury' tier
-- for products above 100000
```

## Challenge 3: Employee Tenure Classification

```
-- Classify employees by tenure
SELECT employee_id,
       INITCAP(first_name) AS name, role,
       EXTRACT(YEAR FROM AGE(joining_date))
       AS years,
       CASE
         WHEN EXTRACT(YEAR FROM
           AGE(joining_date)) >= 5
         THEN 'Veteran'
         WHEN EXTRACT(YEAR FROM
           AGE(joining_date)) >= 2
         THEN 'Experienced'
         ELSE 'New Hire'
       END AS tenure_level
FROM stores.employees
ORDER BY joining_date LIMIT 15;
```

## KEY TAKEAWAYS

Simple CASE: match one column against exact values (like switch). Best for status codes, category names.

Searched CASE: evaluate boolean conditions (like if/else). Best for ranges, complex logic.

Evaluates top-to-bottom — first match wins. Always include ELSE to avoid unexpected NULLs.

Can be used in SELECT, WHERE, ORDER BY — and inside aggregate functions (COUNT, SUM, AVG).

CASE inside aggregates = conditional counting/summing. Builds entire dashboards in one query.



## CASE WHEN vs Application Logic

Q: 'Should business logic be in SQL or in the application?'

A: Simple categorizations (price tiers, status labels, salary bands) belong in SQL — they run faster and are reusable across reports. Complex multi-step logic (workflows, state machines, approval chains) belongs in the application. Rule of thumb: if a data analyst needs it in a report, put it in SQL.

# The E-Commerce Dashboard Problem

Flipkart's dashboard shows customer profiles. Some have phone numbers, some don't. Some have middle names, some don't. Showing NULL on the screen looks broken — users think the app is glitchy.

COALESCE replaces NULLs with friendly defaults: 'Not Provided', 'N/A', or 0.

NULLIF does the reverse — creates NULL when you WANT to ignore a value (like dividing by zero).

CAST converts between types — number to text, text to date, etc.

## DEFINITION

# COALESCE

Technical:

COALESCE(value1, value2, ..., valueN) returns the FIRST non-NULL value from the list. If all values are NULL, returns NULL. It is the SQL standard way to handle NULLs with fallback values.

Layman's:

COALESCE is like asking: 'Do you have a mobile number? No? Okay, home number? No? Fine, I will write N/A.' It tries each option in order and uses the first one that exists.

## SYNTAX: COALESCE

```
-- Replace NULL with a default value
SELECT COALESCE(NULL, 'fallback');
    -- 'fallback'
SELECT COALESCE('hello', 'fallback');
    -- 'hello' (already non-NULL)

-- Multiple fallbacks – tries left to right
SELECT COALESCE(NULL, NULL, 'third');
    -- 'third'

-- With numbers
SELECT COALESCE(NULL, 0);    -- 0
SELECT COALESCE(42, 0);     -- 42
```

## PRO TIP

COALESCE is better than CASE WHEN ... IS NULL for simple NULL defaults. COALESCE(phone, 'N/A') is cleaner than CASE WHEN phone IS NULL THEN 'N/A' ELSE phone END. Use COALESCE for simple NULL replacement, CASE WHEN for complex conditions.

## Easy: Clean Customer Display

EASY

SCENARIO: The customer profile page shows ugly 'null' text for missing phone numbers. The frontend team wants 'Not Provided' instead.

TASK: Show customer\_id, name (INITCAP), phone with COALESCE fallback to 'Not Provided', email, and tier with COALESCE fallback to 'Standard'. Show 15 rows.

HINT: Which function returns the first non-NULL value from a list of arguments? Trap: the fallback value must be the SAME data type as the column — wrapping a numeric column with COALESCE(col, 'N/A') causes a type-cast error.

## Answer

✓ ANSWER

```
SELECT customer_id,  
       INITCAP(first_name) || ' ' ||  
       INITCAP(last_name) AS name,  
       COALESCE(phone, 'Not Provided')  
       AS phone,  
       email,  
       COALESCE(tier, 'Standard') AS tier  
FROM customers.customers  
LIMIT 15;
```

## Medium: Multi-Fallback Last Activity

MEDIUM

SCENARIO: The support dashboard needs ONE 'last activity' timestamp per ticket. Use `resolved_date` when the ticket is resolved; for open tickets (`resolved_date` is NULL) fall back to `created_date`; and as a final safety net, `NOW()`. Also classify priority with a CASE label.

TASK: Show `ticket_id`, `status`, `resolved_date`, a `last_activity` column using COALESCE with multiple fallbacks, and a `priority_label` using CASE. Show 15 OPEN tickets (WHERE `resolved_date` IS NULL) — watch the fallback kick in on every row.

HINT: COALESCE accepts MORE than 2 arguments — it walks the list left-to-right until it finds a non-NULL. Trap: order matters in business logic — the highest-priority source must come first, since later arguments only get checked if everything before them is NULL.



## Answer



```
SELECT ticket_id, status, resolved_date,  
       COALESCE(resolved_date,  
                created_date, NOW())  
       AS last_activity,  
       CASE priority  
         WHEN 'Critical' THEN 'Drop everything'  
         WHEN 'High' THEN 'Handle TODAY'  
         WHEN 'Medium' THEN 'This week'  
         ELSE 'Normal queue'    -- Low  
       END AS priority_label  
FROM support.tickets  
WHERE resolved_date IS NULL  
LIMIT 15;  
  
-- Every row shows the fallback working:  
-- resolved_date is NULL -> last_activity = created_date
```

# NULLIF

Technical:

NULLIF(value1, value2) returns NULL if value1 equals value2, otherwise returns value1. Its primary use is preventing division-by-zero errors.

Layman's:

NULLIF says: 'If this value is X, pretend it does not exist (make it NULL).' Most used for: NULLIF(count, 0) before dividing — because dividing by NULL gives NULL (safe), but dividing by zero crashes your query.

## SYNTAX: NULLIF

```
-- Returns NULL if both values are equal
SELECT NULLIF(10, 10); -- NULL (equal!)
SELECT NULLIF(10, 5);  -- 10 (not equal)
SELECT NULLIF('abc', 'abc'); -- NULL
SELECT NULLIF('abc', 'xyz'); -- 'abc'

-- The killer use case: safe division
-- BAD:  SELECT 100 / 0; -- ERROR! Crash!
-- GOOD: SELECT 100 / NULLIF(0, 0); -- NULL

-- In practice - on RetailMart's real summary table:
SELECT summary_date,
       total_revenue / NULLIF(total_orders, 0)
       AS avg_order_value
FROM finance.revenue_summary
ORDER BY summary_date DESC LIMIT 5;
-- If total_orders = 0, returns NULL not error
```

PRO TIP

## PRO TIP

NULLIF + COALESCE combo: `COALESCE(revenue / NULLIF(orders, 0), 0)`. This says: divide revenue by orders (safely — if orders is 0 return NULL), then if the result is NULL, show 0 instead. Two safety nets in one expression.

## Easy: Safe Product Profit Margin

EASY

SCENARIO: The pricing team wants profit margin for each product:  $(\text{price} - \text{cost\_price}) / \text{price} * 100$ . But some products have price = 0 (free samples), causing division-by-zero crashes.

TASK: Show product\_id, product\_name, price, cost\_price, and profit\_margin\_pct using NULLIF(price, 0) to prevent crashes. Round to 1 decimal. Show 15 products ordered by price DESC.

HINT: Division by zero crashes the query — which function turns a specific value (like 0) into NULL? Trap: NULL propagates through math, so the result becomes NULL — that's safer than crashing, but you may still want to wrap the whole expression in COALESCE to return 0 instead.

## Answer

✓ ANSWER

```
SELECT product_id, product_name,  
       price, cost_price,  
       ROUND((price - cost_price) /  
             NULLIF(price, 0) * 100, 1)  
       AS profit_margin_pct  
FROM products.products  
ORDER BY price DESC  
LIMIT 15;
```

## Medium: Margin with Safe Default

MEDIUM

SCENARIO: Extend the margin report — free products (price=0) should show 0% instead of NULL. Also add a CASE to label margins: 'High' (>50%), 'Normal' (20-50%), 'Low' (<20%).

TASK: Show product\_id, product\_name, price, margin\_pct (NULLIF + COALESCE for default 0), and margin\_label (CASE on the margin). Show 20 products.

HINT: You need TWO defenses — one to handle a zero price, another to handle the resulting NULL. Trap: you can't reference a SELECT alias inside another SELECT expression on the same level — you'll have to repeat the calculation in the CASE, or wrap the whole thing in a derived table.

## Answer

 ANSWER

```
SELECT product_id, product_name,  
       price,  
       COALESCE(ROUND(  
         (price - cost_price) /  
         NULLIF(price, 0) * 100, 1  
       ), 0) AS margin_pct,  
       CASE  
         WHEN COALESCE(ROUND(  
           (price - cost_price) /  
           NULLIF(price, 0) * 100, 1  
         ), 0) > 50 THEN 'High Margin'  
         WHEN COALESCE(ROUND(  
           (price - cost_price) /  
           NULLIF(price, 0) * 100, 1  
         ), 0) > 20 THEN 'Normal Margin'  
         ELSE 'Low Margin'  
       END AS margin_label  
FROM products.products  
ORDER BY price DESC  
LIMIT 20;
```



## GREATEST / LEAST

Technical:

GREATEST(a, b, ...) returns the largest of its arguments; LEAST(a, b, ...) the smallest — compared ACROSS one row, not down a column (that is MIN/MAX, coming on Aggregates day). NULL arguments are ignored in PostgreSQL; only if ALL arguments are NULL is the result NULL.

Layman's:

You have two numbers side by side in the SAME row and want the bigger (or smaller) one — like 'charge whichever is higher: actual price or minimum fee'. GREATEST/LEAST pick per row, instantly.

## GREATEST & LEAST on RetailMart

-- Per product: which is higher, price or cost?

```
SELECT product_id, price, cost_price,  
       GREATEST(price, cost_price) AS higher_side,  
       LEAST(price, cost_price)   AS lower_side  
FROM products.products LIMIT 10;
```

-- Cap a fee: charge 2% of order value, but never more than 500

```
SELECT order_id, net_total,  
       LEAST(net_total * 0.02, 500) AS shipping_fee  
FROM sales.orders LIMIT 10;
```

-- Floor at zero: discounts can never be negative

```
SELECT order_id, discount_amount,  
       GREATEST(discount_amount, 0) AS safe_discount  
FROM sales.orders LIMIT 10;
```

-- LEAST = 'cap at', GREATEST = 'floor at'. Memorize that pairing.

PRO TIP

## PRO TIP

Interview one-liner: GREATEST/LEAST compare values ACROSS a row; MIN/MAX aggregate DOWN a column. Mixing them up is a classic screening-question trap.

# CAST

Technical:

CAST(expression AS target\_type) converts a value from one data type to another. PostgreSQL also supports the shorthand :: operator. Common conversions: TEXT ↔ INTEGER, TEXT ↔ DATE, NUMERIC ↔ INTEGER, TIMESTAMP ↔ DATE.

Layman's:

CAST is like changing the format of data. Convert the number 42 to the text '42' so you can concatenate it. Convert the text '2025-01-15' to a proper DATE so you can do date math. It does not change the stored data — only the output.

## SYNTAX: CAST and :: (PostgreSQL shorthand)

```
-- SQL Standard: CAST(value AS type)
SELECT CAST(42 AS TEXT);           -- '42'
SELECT CAST('42' AS INTEGER);     -- 42
SELECT CAST('2025-01-15' AS DATE);
SELECT CAST(99.99 AS INTEGER);    -- 99 (truncates!)

-- PostgreSQL shorthand: value::type
SELECT 42::TEXT;                  -- '42'
SELECT '42'::INTEGER;            -- 42
SELECT '2025-01-15'::DATE;
SELECT 99.99::INTEGER;           -- 99

-- Both do the same thing
-- :: is shorter and preferred in PostgreSQL
```

## CAST Truncation — Silent Data Loss

The Mistake:

`CAST(99.99 AS INTEGER)` returns 99 — the decimal part is silently dropped. No error, no warning. Your report shows wrong numbers.

The Fix:

Use `ROUND(value)` before casting: `ROUND(99.99)::INTEGER` gives 100. Or use `NUMERIC` for precision: `value::NUMERIC(10,2)`. Always test `CAST` with sample data before applying to reports.

## Easy: Product Codes with Type Conversion

EASY

SCENARIO: The frontend team needs product IDs displayed as 'PROD-1234' and formatted prices as 'Rs. X,XXX' for the catalog page. Currently `product_id` is an integer — concatenating it directly with text causes a type error.

TASK: Show `product_id`, 'PROD-' || `product_id::TEXT` as `product_code`, `product_name`, and 'Rs. ' || `TO_CHAR(price, 'FM99,99,999')` as `display_price`. Show 15 products.

HINT: Two conversions needed — one to allow concatenation of a number with text, one to format the price with thousand separators. Trap: || on a number+string without an explicit cast throws a type error; integers need to become TEXT first.

## Answer

✓ ANSWER

```
SELECT product_id,  
       'PROD-' || product_id::TEXT  
       AS product_code,  
       product_name,  
       'Rs. ' || TO_CHAR(price,  
       'FM99,99,999') AS display_price  
FROM products.products  
ORDER BY price DESC  
LIMIT 15;
```



## Medium: Formatted Order Summary Line

MEDIUM

SCENARIO: Build a formatted order report. Each line should look like: 'Order #12345 — Rs. 15,000 — 15-Jan-2025 — Delivered'. This requires converting numbers and dates to text for concatenation.

TASK: Build a single summary\_line column that concatenates order\_id (as text), formatted amount (TO\_CHAR), formatted date (TO\_CHAR), and status (INITCAP). Show 15 orders from 2025.

HINT: Three formatting decisions — integer to text, number to currency string, date to a human-readable format. Trap: TO\_CHAR with numbers adds a leading space for the sign by default; a 'FM' prefix in the format string suppresses that padding.

## Answer

 ANSWER

```
SELECT
  'Order #' || order_id::TEXT ||
  ' - Rs. ' || TO_CHAR(net_total,
    'FM99,99,999') ||
  ' - ' || TO_CHAR(order_date,
    'DD-Mon-YYYY') ||
  ' - ' || INITCAP(order_status)
  AS summary_line
FROM sales.orders
WHERE order_date >= '2025-01-01'
ORDER BY order_date DESC
LIMIT 15;
```

# Hard: Complete Customer Health Report

HARD

SCENARIO: Build a customer profile card combining ALL Conditional Logic tools. For each customer: a customer code, clean name, phone status, account age category, and tier label.

TASK: Use CAST for customer code ('CUST-' || id::TEXT), COALESCE for phone ('N/A' if NULL), CASE for age category (5+ yrs = 'Veteran', 2-4 = 'Loyal', else = 'New'), and CASE for tier label. Show 20 customers ordered by registration\_date.

HINT: AGE() produces an interval between dates — pair it with EXTRACT to pull out just the year part. Trap: AGE() with one argument compares to TODAY; with two arguments it compares between them. Pick the form that matches your business question.

## Answer (1/2)

✓ ANSWER

```
SELECT
  'CUST-' || customer_id::TEXT
    AS cust_code,
  INITCAP(first_name) || ' ' ||
    INITCAP(last_name) AS name,
  COALESCE(phone, 'N/A') AS phone,
  CASE
    WHEN EXTRACT(YEAR FROM
      AGE(registration_date)) >= 5
      THEN 'Veteran'
    WHEN EXTRACT(YEAR FROM
      AGE(registration_date)) >= 2
      THEN 'Loyal'
    ELSE 'New'
  END AS age_category,
  CASE tier
    WHEN 'Gold' THEN 'Premium'
    WHEN 'Silver' THEN 'Valued'
    WHEN 'Bronze' THEN 'Active'
    ELSE 'Standard'
  END AS tier_label
FROM customers.customers
```

## Answer (2/2)

 ANSWER

```
ORDER BY registration_date  
LIMIT 20;
```

# Crazy: Loyalty Tier From Points (CASE Showcase)

CRAZY

SCENARIO: The CMO wants to see, for each loyalty member, the tier they DESERVE based purely on their points\_balance — computed from ONE table (loyalty.members), no joins (those come later). Bands: 5000+ Platinum, 2000+ Gold, 500+ Silver, else Bronze.

TASK: From loyalty.members alone, show the member, their points, the deserved tier (CASE on points\_balance), how many points to the next tier (CASE), and progress toward Platinum as a percent string. Use COALESCE, NULLIF and CAST.

HINT: This combines all four Conditional Logic tools — CASE for the derived tier and the gap, COALESCE for a safe points value, NULLIF to dodge division by zero, CAST for clean string output. (On the Joins day you will swap in the member's NAME and ACTUAL tier name from the other tables.)

## Answer

✓ ANSWER

```
SELECT
  customer_id,
  COALESCE(points_balance, 0) AS points,
  CASE
    WHEN points_balance >= 5000 THEN 'Platinum'
    WHEN points_balance >= 2000 THEN 'Gold'
    WHEN points_balance >= 500  THEN 'Silver'
    ELSE 'Bronze'
  END AS deserved_tier,
  CASE
    WHEN points_balance >= 5000 THEN 0
    WHEN points_balance >= 2000 THEN 5000 - points_balance
    WHEN points_balance >= 500  THEN 2000 - points_balance
    ELSE 500 - points_balance
  END AS points_to_next_tier,
  CAST(ROUND(100.0 * points_balance
    / NULLIF(5000, 0), 1) AS TEXT) || '%'
    AS pct_of_platinum
FROM loyalty.members
ORDER BY points_balance DESC LIMIT 20;
```

# Hands-On Practice!

Run each query on your RetailMart database. Modify the values, add your own columns. The best way to learn is to experiment.



## Challenge 1: Employee Directory with Defaults

```
-- Build a clean employee display
SELECT employee_id,
       INITCAP(first_name) || ' ' ||
       INITCAP(last_name) AS name,
       COALESCE(role, 'Unassigned') AS role,
       'EMP-' || LPAD(employee_id::TEXT,
       5, '0') AS emp_code,
       salary::TEXT || '/month' AS salary_display
FROM stores.employees LIMIT 15;

-- YOUR TURN: Add a CASE for salary band
```

## Challenge 2: Product Price Display

```
-- Format product prices for catalog
SELECT product_id,
       product_name,
       'Rs. ' || TO_CHAR(price,
       'FM99,99,999') AS display_price,
       CASE
         WHEN cost_price > 0 THEN
           ROUND((price - cost_price) /
           NULLIF(price, 0) * 100)::TEXT
           || '% margin'
         ELSE 'N/A'
       END AS margin_info
FROM products.products
WHERE price > 5000
ORDER BY price DESC LIMIT 15;
```

## Forgetting ELSE — Silent NULLs

The Mistake:

CASE WHEN price > 10000 THEN 'Expensive' END — products with price <= 10000 get NULL instead of a label. No error, just blank cells in your report.

The Fix:

Always add ELSE: CASE WHEN price > 10000 THEN 'Expensive' ELSE 'Affordable' END. Without ELSE, unmatched rows silently become NULL.

## NULL = NULL is NOT TRUE

The Mistake:

CASE WHEN phone = NULL THEN 'Missing' END — this NEVER matches because NULL = NULL is UNKNOWN, not TRUE.  
No rows ever get labeled 'Missing'.

The Fix:

Use IS NULL: CASE WHEN phone IS NULL THEN 'Missing' ELSE phone END. Or better yet: COALESCE(phone, 'Missing').

## COALESCE Type Mismatch

The Mistake:

`COALESCE(price, 'N/A')` — `price` is `NUMERIC`, `'N/A'` is `TEXT`. PostgreSQL throws a type error because all `COALESCE` arguments must be the same type.

The Fix:

Cast to the same type: `COALESCE(price::TEXT, 'N/A')`. Or use `CASE WHEN`: `CASE WHEN price IS NULL THEN 'N/A' ELSE price::TEXT END`.

## CASE WHEN with OR vs Multiple WHENs

The Mistake:

Using OR incorrectly: `CASE WHEN status = 'Delivered' OR 'Shipped' THEN 'Done' END` — this is a syntax error. 'Shipped' alone is not a boolean expression.

The Fix:

Either: `CASE WHEN status = 'Delivered' OR status = 'Shipped' THEN 'Done' END`. Or cleaner: `CASE WHEN status IN ('Delivered', 'Shipped') THEN 'Done' END`.

## Simple CASE vs Searched CASE

Q: 'What is the difference between Simple CASE and Searched CASE?'

A: Simple CASE matches ONE expression against values: CASE status WHEN 'Active' THEN ... Good for equality checks on a single column. Searched CASE evaluates boolean conditions: CASE WHEN price > 1000 THEN ... Good for ranges, comparisons across columns, and complex logic.

## COALESCE vs IFNULL vs NVL

Q: 'What is COALESCE and how does it differ from IFNULL?'

A: COALESCE is SQL standard — works in PostgreSQL, MySQL, SQL Server, Oracle. It accepts N arguments and returns the first non-NULL. IFNULL is MySQL-only (2 arguments). NVL is Oracle-only. ISNULL is SQL Server-only. Always use COALESCE for portability and flexibility.



## Prevent Division by Zero

Q: 'How do you handle division by zero in SQL?'

A: Use NULLIF: `revenue / NULLIF(order_count, 0)`. NULLIF returns NULL when `order_count` is 0, and anything divided by NULL gives NULL instead of an error. Optionally wrap with COALESCE for a default: `COALESCE(revenue / NULLIF(order_count, 0), 0)`.

## CASE WHEN NULL Gotcha

Q: 'What does CASE WHEN column = NULL return?'

A: It NEVER matches because `NULL = NULL` evaluates to `UNKNOWN` (not `TRUE`). To check for `NULL` inside `CASE`, use `IS NULL`: `CASE WHEN column IS NULL THEN 'missing' END`. This is one of the most common SQL interview traps.

## Conditional Aggregation

Q: 'How do you count only certain rows within an aggregate?'

A: Use CASE inside COUNT: COUNT(CASE WHEN order\_status = 'Delivered' THEN 1 END). For sums: SUM(CASE WHEN category = 'Electronics' THEN amount END). Non-matching rows return NULL, which is skipped by aggregate functions. This creates pivot-style reports in a single query.

## CAST vs TO\_CHAR vs ::TEXT

Q: 'When do you use CAST vs TO\_CHAR for converting numbers to text?'

A: CAST(42 AS TEXT) or 42::TEXT gives '42' — plain conversion. TO\_CHAR(42000, 'FM99,999') gives '42,000' — formatted with commas. Use CAST/:: for simple type conversion. Use TO\_CHAR when you need specific formatting (commas, decimals, currency).

## CASE WHEN vs DECODE

Q: 'What is DECODE and how does it compare to CASE?'

A: DECODE is Oracle-only. It works like Simple CASE: `DECODE(status, 'A', 'Active', 'I', 'Inactive', 'Unknown')`. CASE WHEN is SQL standard — works in PostgreSQL, MySQL, SQL Server, Oracle. Always prefer CASE WHEN for portability. Plus, CASE supports conditions (ranges, LIKE, IN) that DECODE cannot do.

## Real Interview: Classify and Count

Q: 'Write a query to show total orders, and what percentage are high-value (> 10000), medium (5000-10000), and low (< 5000).'

A: This combines Conditional Logic + Aggregates & Grouping: `ROUND(100.0 * COUNT(CASE WHEN net_total > 10000 THEN 1 END) / COUNT(*), 1) AS high_pct`. Repeat for medium and low tiers. The interviewer tests if you know CASE inside aggregates — the most powerful pattern in analytics SQL.

## KEY TAKEAWAYS

### CASE WHEN:

1. Simple CASE: match one column against fixed values (like switch). Good for status mapping.
2. Searched CASE: evaluate boolean conditions (like if/else). Good for ranges, complex logic.
3. Evaluates top-to-bottom — first match wins. Always include ELSE.
4. Can be used in SELECT, WHERE, ORDER BY, and inside aggregates (conditional counting/summing).

### NULL HANDLING & TYPE CONVERSION:

5. COALESCE(a, b, c) — returns first non-NULL. Essential for clean reports.
6. NULLIF(a, b) — returns NULL if a = b. Essential for safe division.
7. CAST(x AS TYPE) or x::TYPE — convert between data types.
8. Pattern: SUM(CASE WHEN condition THEN amount END) = conditional aggregation.

## Conditional Logic & Derived Columns

### -- Simple CASE

```
CASE column
  WHEN 'val1' THEN 'result1'
  WHEN 'val2' THEN 'result2'
  ELSE 'default'
END AS alias
```

### -- Searched CASE

```
CASE
  WHEN condition1 THEN 'result1'
  WHEN condition2 THEN 'result2'
  ELSE 'default'
END AS alias
```

### -- CASE in Aggregates

```
COUNT(CASE WHEN status = 'Done' THEN 1 END)
SUM(CASE WHEN cat = 'X' THEN amt END)
AVG(CASE WHEN price > 1000 THEN price END)
```



## Conditional Logic & Derived Columns

### -- COALESCE

```
COALESCE(phone, 'N/A')  -- NULL → 'N/A'  
COALESCE(a, b, c, 0)    -- first non-NULL
```

### -- NULLIF

```
NULLIF(count, 0)        -- 0 → NULL  
total / NULLIF(count, 0) -- safe division
```

### -- CAST & ::

```
CAST(price AS INTEGER)  
price::INTEGER          -- PG shorthand  
'2025-01-15'::DATE  
42::TEXT                -- for concatenation
```

## CHALLENGE

# Conditional Logic Take-Home Challenge (1/3)

Build these queries on RetailMart V3:

## Conditional Logic Take-Home Challenge (2/3)

- 1.: Classify all orders as 'High Value' (> 10000), 'Medium' (5000-10000), 'Low' (< 5000). Show order\_id, net\_total, and the classification.
- 2.: Map order statuses to user-friendly labels using Simple CASE. Sort by custom priority (Pending first) using CASE in ORDER BY.
- 3.: Show all customers with COALESCE on phone ('N/A') and tier ('Standard'). Include name (INITCAP) and account age.
- 4.: Build a customer profile: 'CUST-' || id, name, phone status (CASE: Has Phone / Missing), tier label (CASE), and registration year.
- 5.: Create a single-row dashboard: total orders, delivered count, pending count, cancelled count, avg delivered value — using CASE inside aggregates.
- 6.: Build an order summary line: 'Order #ID — Rs. X,XXX — DD-Mon-YYYY — Status' using CAST, TO\_CHAR, and || concatenation.

## CHALLENGE

# Conditional Logic Take-Home Challenge (3/3)

Push your SQL file to GitHub!

<https://github.com/starlordali4444/postgresql>

## WHAT YOU ACHIEVED TODAY

Your queries can now THINK. CASE WHEN adds decision-making. COALESCE handles NULLs gracefully. NULLIF prevents crashes. CAST converts between types. Combined with earlier sessions skills, you can now build real analytics dashboards.

Tomorrow: Aggregates — COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING. You will learn to SUMMARIZE entire tables into business reports. Combined with today's CASE WHEN, you will build powerful pivot-style reports.

# Coming Tomorrow

How many customers per city? What is the total revenue per month? Which store has the highest average order value?

GROUP BY + aggregate functions (COUNT, SUM, AVG, MIN, MAX) turn millions of rows into concise business reports.

HAVING filters groups (WHERE filters rows, HAVING filters groups).

Aggregates & Grouping is where SQL becomes a REPORTING POWERHOUSE. Combined with CASE WHEN, you will create dashboards that executives rely on.